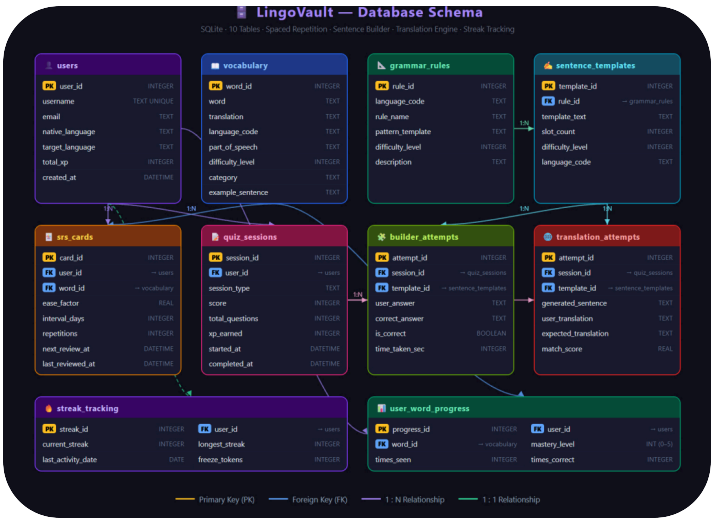

"I am wanting to build my project according to this schema and.
now can u build this app for me. just remember that i am wanting to keep most of the things in my app offline based meaning user signup and login
and basic features like streak completetion
level these things will be in database. else other all the things like language and stuff everything will be hard coded.
can u build?"
I was going to give this prompt in antigravity ai. but i will not give just direct this prompt. now u just make a full documentation for my app so that i can give that into antigravity (so that it can understand each things easily and make my full app easily.

Got it — you don't need code yet, you need **clear, structured product + technical documentation** that another AI (like Antigravity) can execute cleanly. I'll write this like a proper spec so it's unambiguous and implementation-ready.

📘 LingoVault — Full App Documentation

1. 🧠 App Overview

LingoVault is an **offline-first language learning app** focused on:

- Spaced repetition (SRS)
- Sentence building
- Translation practice
- Grammar-based learning
- Streak & XP progression

Core Philosophy

- **Offline-first**: No server dependency for core usage
- **Local database (SQLite)** handles all user data
- **Hardcoded language content** (vocabulary, grammar, templates)
- Lightweight, fast, and deterministic behavior

---

2. 🏗️ System Architecture

2.1 High-Level Structure

```
App
 ├── UI Layer
 ├── Business Logic Layer
 ├── Local Database (SQLite)
 └── Static Content (hardcoded JSON / constants)
```

2.2 Data Sources

Type	Source
User data	SQLite
Progress tracking	SQLite
Vocabulary	Hardcoded
Grammar rules	Hardcoded
Sentence templates	Hardcoded

3. 🗄️ Database Design (SQLite)

Use the schema you provided exactly.

Tables Summary

3.1 `users`

Stores local user identity.

- No real authentication (offline only)
- Just username-based login

3.2 `srs_cards`

Tracks spaced repetition state per word.

Key logic:

- `ease_factor`
- `interval_days`
- `repetitions`
- `next_review_at`

3.3 `quiz_sessions`

Tracks a learning session.

Types:

- `srs_review`
- `sentence_builder`
- `translation`

3.4 `builder_attempts`

Tracks sentence construction attempts.

3.5 `translation_attempts`

Tracks translation exercises.

3.6 `streak_tracking`

Tracks daily engagement.

3.7 `user_word_progress`

Tracks mastery of each word.

---

4. 📦 Static (Hardcoded) Data

These are NOT stored in DB:

4.1 Vocabulary

Stored as JSON or constants:

```
{
  "word_id": 1,
  "word": "apple",
  "translation": "manzana",
  "language_code": "es",
  "part_of_speech": "noun",
  "difficulty_level": 1,
  "category": "food",
```

‹/› JSON

4.2 Grammar Rules

```
{
  "rule_id": 1,
  "rule_name": "Present Tense",
  "pattern_template": "Subject + Verb(base)",
```

‹/› JSON

4.3 Sentence Templates

```
{
  "template_id": 1,
  "rule_id": 1,
  "template_text": "I ___ an apple",
```

JSON

---

5. 🔑 Core Features

5.1 👤 User System (Offline)

Signup

- Input: username only
- Stored in `users` table

Login

- Select existing username
- No password required

5.2 🔥 Streak System

Logic:

- If user opens app today → increment streak
- If missed a day → reset streak

Fields:

- `current_streak`
- `longest_streak`
- `last_activity_date`

Optional:

- `freeze_tokens` to prevent loss

5.3 ⭐ XP System

XP is stored in:

- `users.total_xp`
- `quiz_sessions.xp_earned`

XP Sources:

Action	XP
Correct SRS answer	+5
Correct sentence	+10
Perfect translation	+15

5.4 📚 SRS (Spaced Repetition)

Algorithm (simplified SM-2):

```
if correct:
    repetitions += 1
    interval_days *= ease_factor
    ease_factor += 0.1
else:
    repetitions = 0
    interval_days = 1
    ease_factor -= 0.2

next_review_at = today + interval_days
```

5.5 🧩 Sentence Builder

Flow:

1. Pick a sentence template
2. Fill slots with correct words
3. Compare with correct answer

Stored in:

- `builder_attempts`

5.6 🌐 Translation Engine

Flow:

1. Show generated sentence
2. User translates
3. Compare with expected translation

Matching:

- Simple string match OR
- Fuzzy similarity score

Stored in:

- `translation_attempts.match_score`

5.7 📊 Word Mastery

Tracked in:

- `user_word_progress`

Logic:

```
mastery_level = min(5, times_correct / times_seen * 5)
```

---

6. 🔄 App Flow

6.1 First Launch

```
→ Create user
→ Initialize streak
→ Load static data
```

6.2 Daily Usage Flow

```
Open App
 → Update streak
 → Show dashboard
 → Start session
     → SRS / Builder / Translation
 → Save progress
 → Update XP
```

---

7. 🧠 Business Logic Rules

7.1 Session Creation

- Each activity = one `quiz_session`

ChatGPT Get Plus ×

7.2 Answer Evaluation

Type	Logic
SRS	Exact match
Builder	Template validation
Translation	String similarity

7.3 Progress Updates

- Update `user_word_progress`
- Update SRS card
- Add XP
- Update streak

---

8. 🖥️ UI Requirements

Screens:

1. Splash / Login

- Select or create user

2. Dashboard

- Streak
- XP
- Start session buttons

3. SRS Screen

- Word → user answers → feedback

4. Sentence Builder

- Fill blanks UI

5. Translation Screen

- Input text field

6. Results Screen

- Score
- XP gained

---

9. ⚙️ Technical Constraints

- Must work **100% offline**
- Use **SQLite**
- No external APIs
- No authentication server
- Fast local queries

---

10. 🧪 Edge Cases

- User skips day → streak reset
- No due SRS cards → fallback to new words
- Empty session → prevent completion
- Typo handling in translation

---

11. 🚀 Future Extensions (Optional)

- Cloud sync

- Multiple languages
- Audio pronunciation
- AI-generated sentences

---

12. 🧩 Implementation Notes for Antigravity AI

Tell it clearly:

- Follow schema exactly
- Do NOT create backend server
- Use local SQLite only
- Treat vocabulary & grammar as static data
- Focus on deterministic logic (no randomness unless defined)

---

13. 📌 Final Instruction Prompt (You can paste this)

You can prepend this when giving to Antigravity:

> Build a fully offline-first language learning app called "LingoVault" using SQLite for all persistent data. Follow the provided schema strictly. All vocabulary, grammar rules, and sentence templates must be hardcoded (not stored in DB). Implement SRS, sentence builder, translation exercises, streak tracking, and XP system exactly as described in the documentation. No backend or API usage is allowed.

If you want next step, I can:

- ↳ Turn this into **API-level spec**
- Or ↳ generate **full folder structure + code starter**
- Or ↳ optimize this for **Flutter / React Native / Web**

Just tell me 👍